

BHTTP/1: a binary framing for HTTP

Specification, version 1

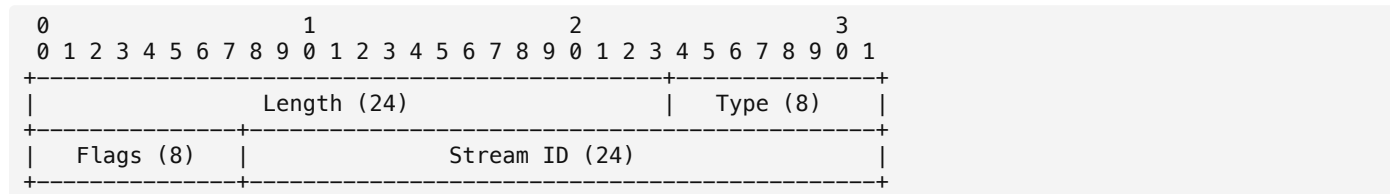
MUST, MUST NOT, SHOULD and MAY are used as in RFC 2119. Integers are unsigned and big-endian. Sizes are in bytes.

1. Connection

BHTTP/1 runs over one TCP connection (default port 9000) with no preface. The client sends requests on it and the server answers each one, in order, on the same connection, which stays open until the client closes it. A client MUST NOT open a second connection during one run (one set of requests). If the connection closes early, the client stops and reports an error rather than reconnecting. A client SHOULD wait for the end of a response before sending the next request. A server MUST still accept requests sent earlier (pipelining) and answer them in order. A server reads a request completely before answering it. A server MAY close a connection idle for at least 10 s, or one whose request or response stops moving.

2. Frame header

Every frame is an 8-byte header followed by Length bytes of payload.



- Length: payload size, not counting the header (0 to 16 777 215).
- Type: 0x00 DATA, 0x01 HEADERS. Any other value is an unknown type (§3).
- Flags: 0x01 END_STREAM marks the last frame of a request or response. Senders set the other bits to 0 and receivers ignore them.
- Stream ID: the client numbers its requests on a connection, starting at 1. Each request MUST use a higher ID than the one before (gaps are allowed), and the response carries the same ID. Stream 0 stands for the connection itself: in v1 it carries only unknown frames and the connection-error response of §6. After using ID 16 777 215 the client MUST close the connection.

3. Unknown frame types

A receiver that reads a frame of unknown type MUST skip it: read and discard exactly Length payload bytes, then go on to the next frame. It MUST NOT reply to it, close the connection over it, or let it change any request state, whatever its stream ID or flags (END_STREAM on an unknown frame ends nothing). This holds everywhere in the frame sequence, including between the frames of one request or response. Types 0xF0–0xFF will never be assigned, so senders can use them to test that peers skip. HTTP/2 has the same rule outside its header blocks (RFC 9113 §4.1, §5.5).

4. Messages

A request is one HEADERS frame and then zero or more DATA frames on the same stream, with END_STREAM on the last frame. A response has the same shape. A message with no body is a single HEADERS frame with END_STREAM set. The header block MUST fit in one HEADERS frame of at most 65 536 bytes; a receiver MUST accept any block within that size, however many fields it holds. Senders SHOULD keep DATA frames to 16 384 bytes or less; receivers MUST accept any Length. A body ends at END_STREAM. When a response that is not to HEAD carries content-length, its DATA MUST add up to exactly that many bytes, and the client treats any difference as a protocol error. A sender that cannot finish a body (for example, a file read fails) MUST close the connection without sending END_STREAM, so a cut-off body is never mistaken for a whole one.

5. Header block

A HEADERS payload is a sequence of fields with no count and no terminator; the frame's Length bounds it. In the grammar, tag is one byte and len is one or two bytes.

field	=	tag	[name]	value		;	name present only when tag = 0
name	=	len	bytes		value = len	bytes	
len	=	0xxxxxxx				;	0 to 127
		1xxxxxxx xxxxxxxx				;	0 to 32 767 (15 bits, big-endian)

Tag 0 means the name follows as a literal. Tags 1 to 10 name the fields of a plain GET exchange, listed below. Tags 11 to 255 are not defined in v1.

tag	name	tag	name	tag	name	tag	name	tag	name
1	:method	3	:status	5	user-agent	7	server	9	content-type
2	:path	4	host	6	accept	8	date	10	content-length

- Literal names are 1 to 32 767 lowercase HTTP token characters (RFC 9110 §5.6.2). Pseudo-headers (names starting with :) can therefore only be sent by tag. Senders use the tag for any name in the table. Receivers also accept the other table

names as literals.

- Values are 0 to 32 767 bytes and may hold any byte except NUL, CR and LF. date uses the HTTP date format (RFC 9110 §5.6.7). Senders SHOULD use the one-byte len when it fits; receivers accept both forms.
- Pseudo-headers come before all other fields. :method, :path, :status, host and content-length appear at most once; other names may repeat.
- A request has one :method and one :path, and no :status. The path starts with / and is at most 1024 bytes as sent. host SHOULD be sent.
- A response has one :status, three ASCII digits from 200 to 599, and no :method or :path. v1 has no 1xx responses.
- A block that breaks any of these rules, uses tags 11 to 255, or has a field whose length runs past the end of the payload is malformed.

6. Server

To find the file, the server cuts :path at the first ? or # and decodes %XX escapes (so %2F is a slash and %2e%2e is ..). It refuses any .. segment and hides any other segment that starts with a dot. It then resolves the path under its document root, following symbolic links, and checks that it still lies inside the root. A path that ends in /, or that names a directory, means that directory's index.html. A found file gets 200 with content-type, content-length, server and date, then the file as DATA. Errors get a short text/plain body. A HEAD request gets the same headers as GET, and never DATA. Checks run in the order of the table.

status	when
400	malformed header block (§5), block over 65 536 bytes, bad % escape or %00 in the path
405	a method other than GET or HEAD
403	a .. segment, a path resolving outside the root, or a file the server may not read
404	no regular file at that path, or a segment starting with a dot

These are stream errors: the response goes on the request's stream and the connection stays open. Any request body is read and discarded first. A connection error means the frame sequence cannot be trusted any more. The cases are: HEADERS on stream 0 or with an ID not higher than the last; DATA with no request open or on a different stream; and a new HEADERS before the open request's END_STREAM. Requests completed before the bad frame are answered first. Then the server sends a 400 response on stream 0 with the reason in its body, and closes. If a frame is cut off by EOF the peer has gone, so the server just closes.

7. Client

The client matches responses to its requests in order. These are protocol errors, after which it closes the connection: a frame for any stream other than the oldest unanswered request (except HEADERS on stream 0, which reports a connection error); DATA before HEADERS or after END_STREAM; a second HEADERS in one response; a malformed or oversized response header block; a content-length mismatch; a frame cut off by EOF; and no data for longer than it is willing to wait.

8. Design notes

HTTP/2's header is Length 24, Type 8, Flags 8, then a reserved bit and a 31-bit Stream ID: 9 bytes. Until draft 14 (2014) its Length was 14 bits. When it grew to 24 bits, the old 16 KiB limit stayed as the default SETTINGS_MAX_FRAME_SIZE, which a receiver raises if it will accept bigger frames; small frames also stop one stream holding up the others on a shared connection. Both ends start streams, client streams odd and server streams even, so each side gets 2³⁰ IDs. The top bit is reserved with no meaning (RFC 9113 §4.1). In SPDY, HTTP/2's predecessor, that bit marked control frames.

BHTTP/1 uses 24 / 8 / 8 / 24, which makes 8 bytes.

- Length 24. v1's own limits (16 KiB DATA, 64 KiB header blocks) almost fit 16 bits. The extra byte is headroom for a v2 that wants larger frames. The width cannot be changed later without breaking §3, because skipping depends on every version reading Length the same way.
- Type 8 and Flags 8. v1 uses two types and one flag. §3 lets a v2 add frame types that v1 peers simply skip. New flags and table tags are different: a v1 peer would ignore or reject them. So a v2 may use those only after the peer has agreed (below).
- Stream ID 24. v1 has one request in progress at a time and no server-started streams, so it needs no odd/even split and no reserved bit. The ID matches each response to its request. At 1000 requests a second, 2²⁴ IDs last 4.6 hours on one connection.

HPACK has a 61-entry static table of name-value pairs, a dynamic table, Huffman coding and prefix-coded integers.

BHTTP/1 takes two of its ideas: names from a fixed table (as in HPACK's "literal with indexed name") and length-prefixed literals for everything else. Values are always literal, and the length prefix is a simpler one- or two-byte form. A typical request is 57 bytes on the wire (8-byte header plus a 49-byte block); the same request as HTTP/1.1 text is 85 bytes.

To grow, a v2 peer would first send a new frame type, say SETTINGS, listing what it supports. A v2 server would acknowledge it before its first response; a v1 server skips it. So a v2 client that gets its first response with no acknowledgement knows to stay with v1, without waiting on a timer.